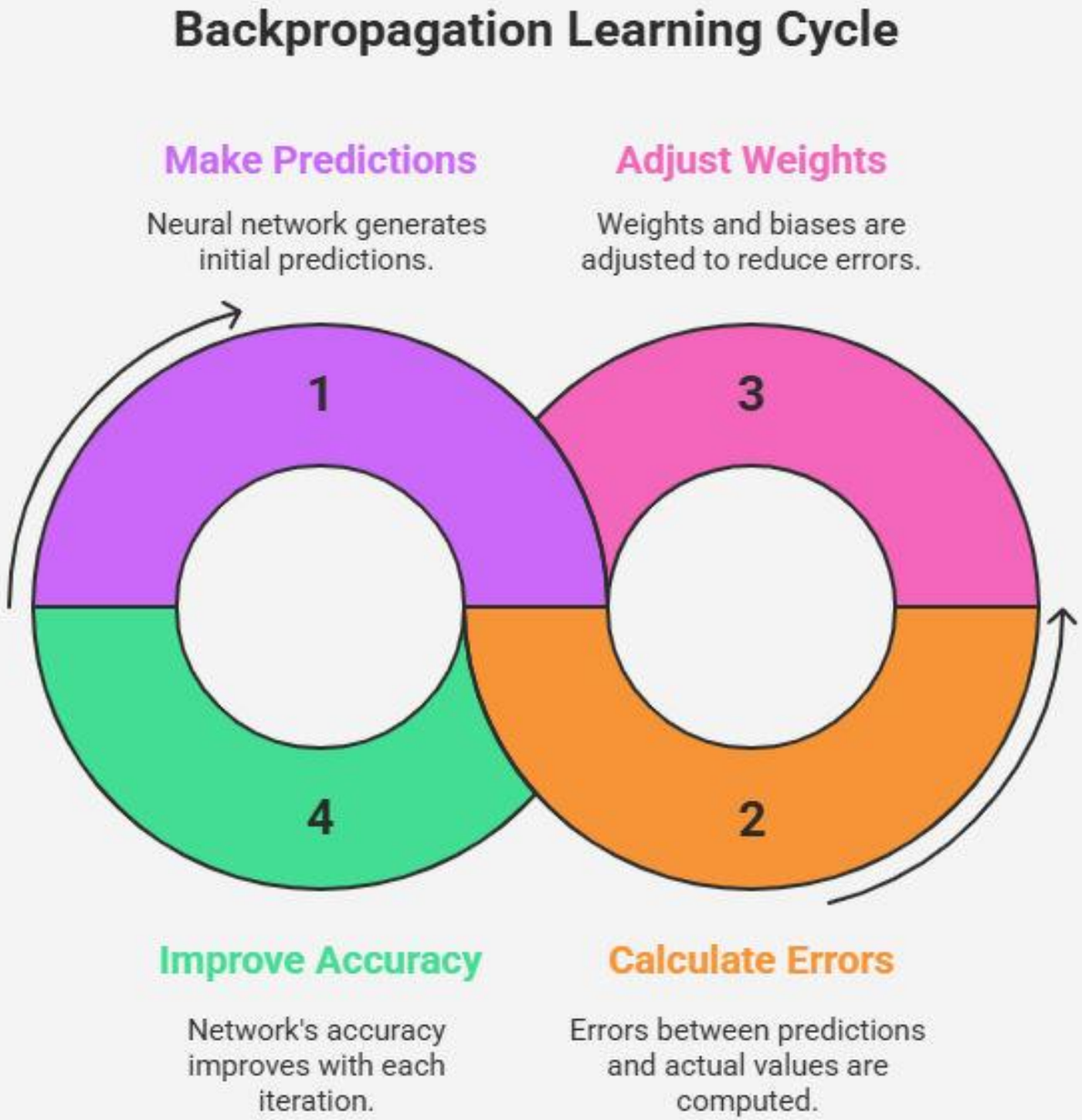
UNIT-I

Multi-Layer Perceptron



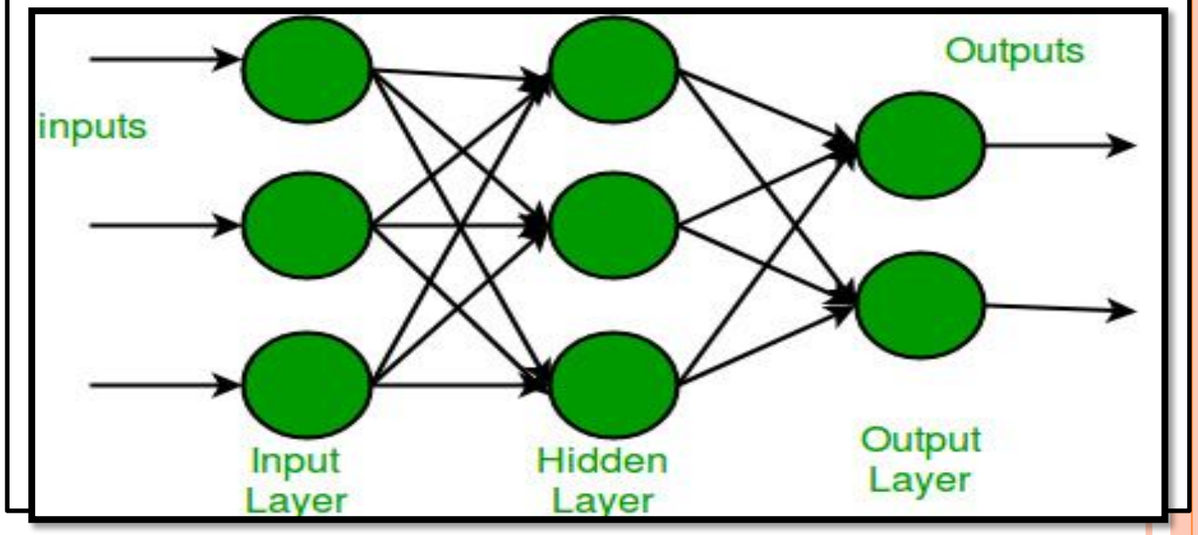
Topics :

- Multi Layer Neural Networks
- Back Propagation Algorithm
- Example Problem
- Advantages
- Applications

Multi Layer Perceptron

- A **fully connected Multi-Layer Neural Network** is called a **Multilayer Perceptron (MLP).**
- The **Multi-Layer-Perceptron** was **first introduced** by **M. Minsky and S. Papert in 1969.** It is an **extended Perceptron** and has **one ore more hidden neuron layers between its input and output layers**
- Due to its extended structure, a **Multi-Layer-Perceptron** is **able to solve every logical operation**, **including the XOR problem.**

- Multilayer perceptron's (MLPs) are **feed forward neural networks trained with** the standard **back propagation algorithm.**
- By using the **Back propagation**, is a procedure to **repeatedly adjust the weights** so as to **minimize the difference between actual output and desired output**



Multi Layer Perceptron

- In the multi-layer perceptron diagram above, we can see that there are **three input nodes** and the **hidden layer** has **three nodes**. The **output layer** gives **two outputs**, therefore there are **two output nodes.**
- The nodes in the **input layer take input** and **forward it for further process,** in the diagram above the nodes in the **input layer forwards their output to each of the three nodes**
- In the **hidden layer**, and in the same way, **the hidden layer processes the information and passes it to the output layer**.

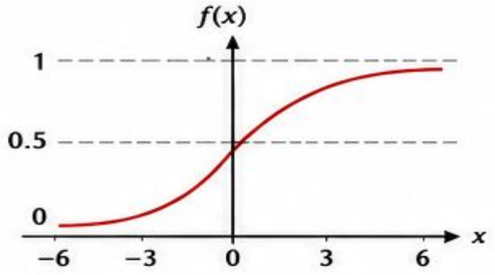
- Hence, **a multi-layered perceptron model** has considered as **artificial neural networks** having **various layers** in which **activation function does not remain linear**, similar to a **single layer perceptron model**.
- In this Networks, we used the **Non Linear Activation Functions used.ie, Sigmoid, Relu, Tanh etc.** Since the probability of any event lies **between 0 and 1.**
- In Multi Layer Networks, We can follow the **Supervised Learning.**

DIFFERENT NON LINEAR ACTIVATION FUNCTIONS

Activation Functions introduce **non-linearity** in neural networks, enabling the model to learn complex patterns.

1. SIGMOID (LOGISTIC)

$$f(x) = \frac{1}{1 + e^{-x}}$$



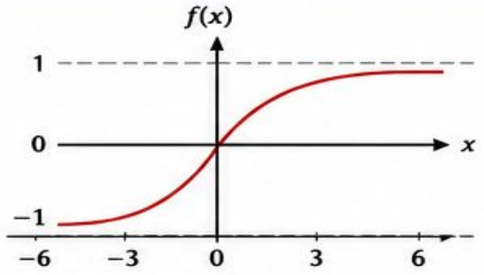
Range : (0, 1)

Used in : Output layer for binary classification

Use : Used for probability output (0 to 1).

2. TANH (HYPERBOLIC TANGENT)

$$f(x) = \tanh(x) = \frac{e^{x} - e^{-x}}{e^{x} + e^{-x}}$$



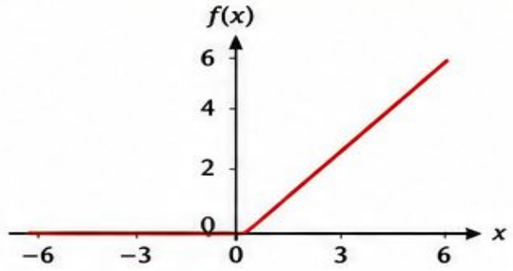
Range : (−1, 1)

Used in : Hidden layers in many neural networks

Use : Used when output can be negative or positive.

3. RELU (RECTIFIED LINEAR UNIT)

$$f(x) = \max(0, x) = \begin{cases} 0, & x \le 0 \\ x, & x > 0 \end{cases}$$



Range : [0, ∞)

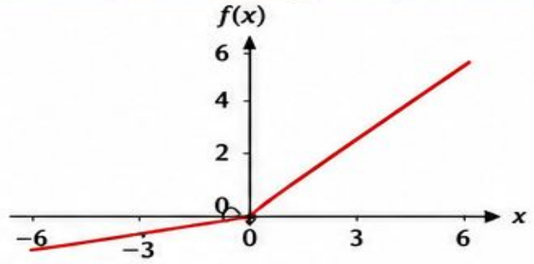
Used in : Hidden layers of deep neural networks

Use : ReLU is widely used in deep learning, especially for **Images (CNNs)**.

4. LEAKY RELU

$$f(x) = \begin{cases} ax, & x \le 0 \\ x, & x > 0 \end{cases}$$

(a is small, e.g., 0.01)



Range : (−∞, ∞)

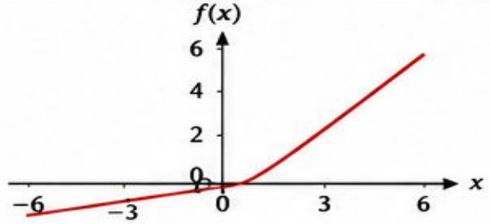
Used in : Improves learning when dying ReLU problem occurs.

Use : Used in deep networks to avoid dead neurons.

5. PARAMETRIC RELU (PReLU)

$$f(x) = \begin{cases} ax, & x \le 0 \\ x, & x > 0 \end{cases}$$

(a is learnable parameter)



Range : (−∞, ∞)

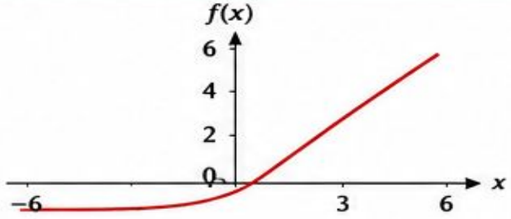
Used in : Better than ReLU as 'a' is learned during training.

Use : Used in deep networks for better performance.

6. ELU (EXPONENTIAL LINEAR UNIT)

$$f(x) = \begin{cases} ax, & x \le 0 \\ x, & x > 0 \end{cases}$$

$(a > 0)$



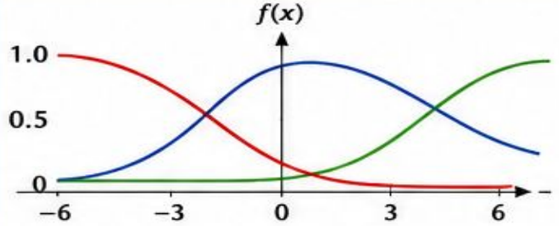
Range : (−∞, ∞)

Used in : Faster and more accurate learning than ReLU.

Use : Used in deep networks to improve learning speed.

7. SOFTMAX (for output layer)

$$f_i(x) = \frac{e^{x_i}}{\sum_{j=1}^{n} e^{x_j}}$$



Range : (0, 1) (Sum of all outputs = 1)

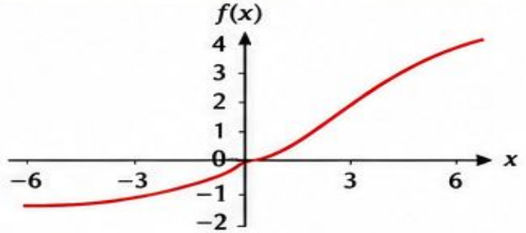
Used in : Output layer for multi-class classification problems.

Use : Converts scores into probabilities for different classes.

8. SWISH

$$f(x) = x \cdot \sigma(x) = \frac{x}{1 + e^{-x}}$$

where $\sigma(x) = \frac{1}{1 + e^{-x}}$



Range : (−∞, ∞)

Used in : Modern deep networks, gives better performance than ReLU.

Use : Used in advanced deep learning models.

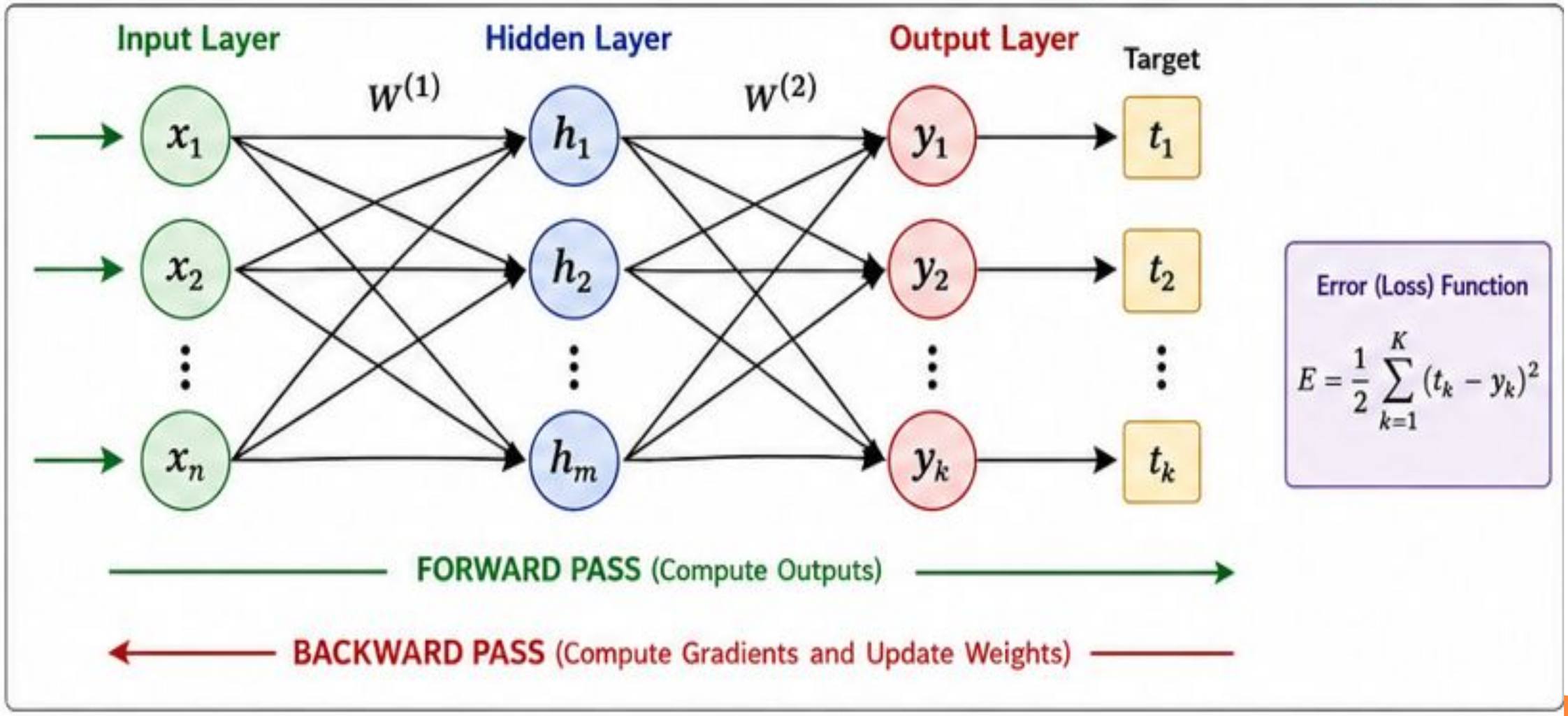
Note : Choosing the right activation function depends on the problem, network architecture, and type of task.

Back Propagation Algorithm

- In an Artificial Neural Network, the **values of weights and biases** are **randomly initialized**.
- Due to **random initialization**, the neural network **probably has errors for the given inputs.**
- So, We need to **reduce error values** as much as possible.
- So, for **reducing these error values,** we need a mechanism that can **compare the desired output of the neural network** with the **Target network's output**

- Suppose, that **consists of errors** and **adjusts its weights and biases** such that it **gets closer to the desired output after each iteration**.
- For this, we **train the network** such that it **back propagates and updates the weights and biases**. This is the concept of the **back propagation algorithm.**
- Below are the steps that an artificial neural network follows to **gain maximum accuracy** and **minimize error values**

BACK PROPAGATION ALGORIHM



How Back Propagation Works?

- **Backpropagation** is a **learning algorithm** used in **deep learning** to **minimize the loss function** by **propagating the output error backward through the network** and **updating weights and biases** using **gradient descent.**

Need for Backpropagation

Backpropagation is used to:

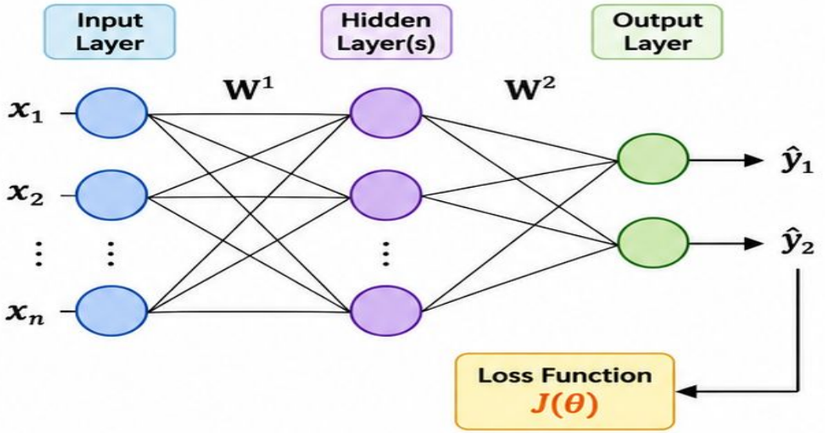
- Train deep neural networks efficiently.
- Reduce prediction error.
- Learn complex nonlinear patterns.
- Optimize millions of network parameters.
- Improve model accuracy.

BACKPROPAGATION

Backpropagation consists of two major phases.

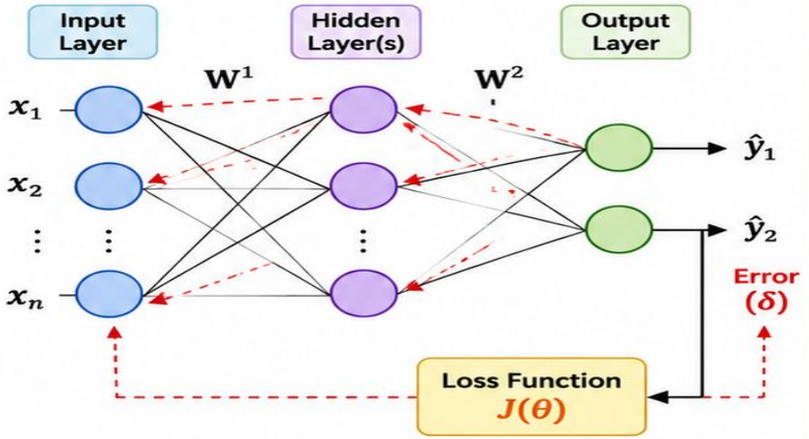
1. FORWARD PROPAGATION

1. Input data is fed into the network.
2. Each neuron computes a weighted sum. Σ
3. Activation functions produce outputs.
4. Final output (prediction) is generated.
5. Loss function calculates prediction error. $J(\theta)$

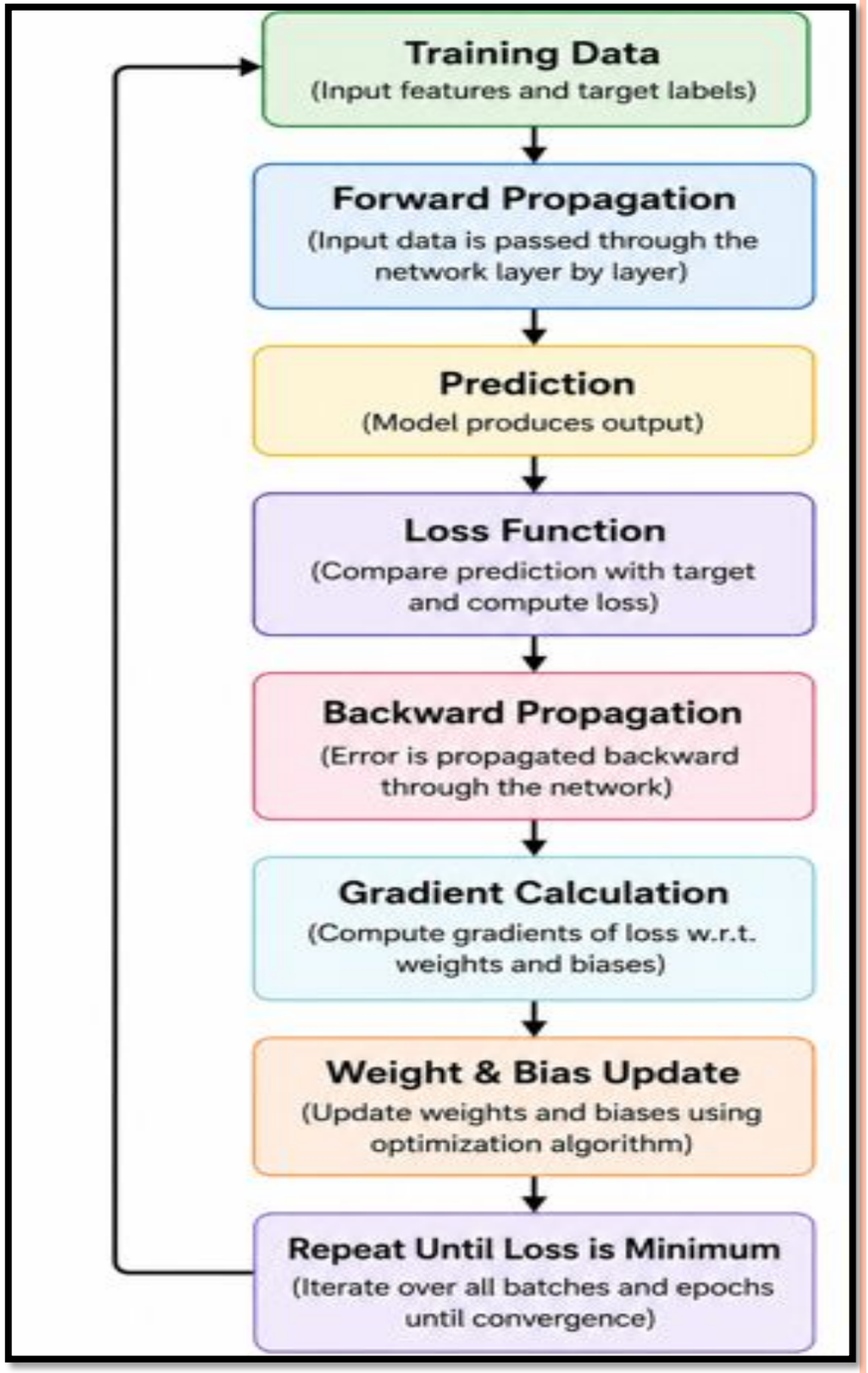


2. BACKWARD PROPAGATION

1. Compute gradients of the loss function. $\frac{\partial J}{\partial \theta}$
2. Propagate errors backward from output to input layer.
3. Calculate gradients for each weight and bias. $\frac{\partial J}{\partial w}$, $\frac{\partial J}{\partial b}$
4. Update weights using Gradient Descent. $w := w - \alpha \frac{\partial J}{\partial w}$
5. Repeat for all epochs until convergence.

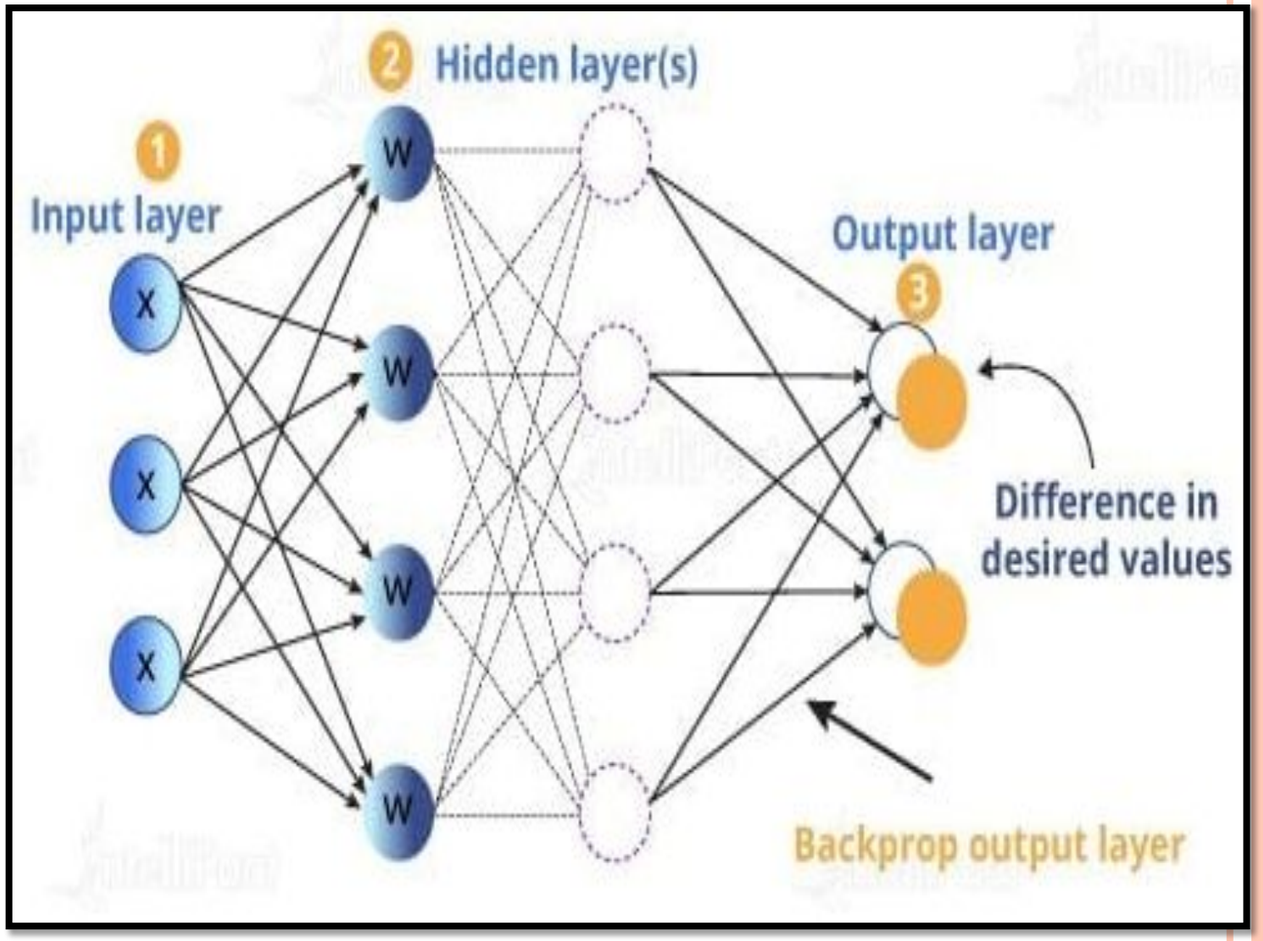


This process is repeated for all training samples and multiple epochs until the loss is minimized and the model converges.

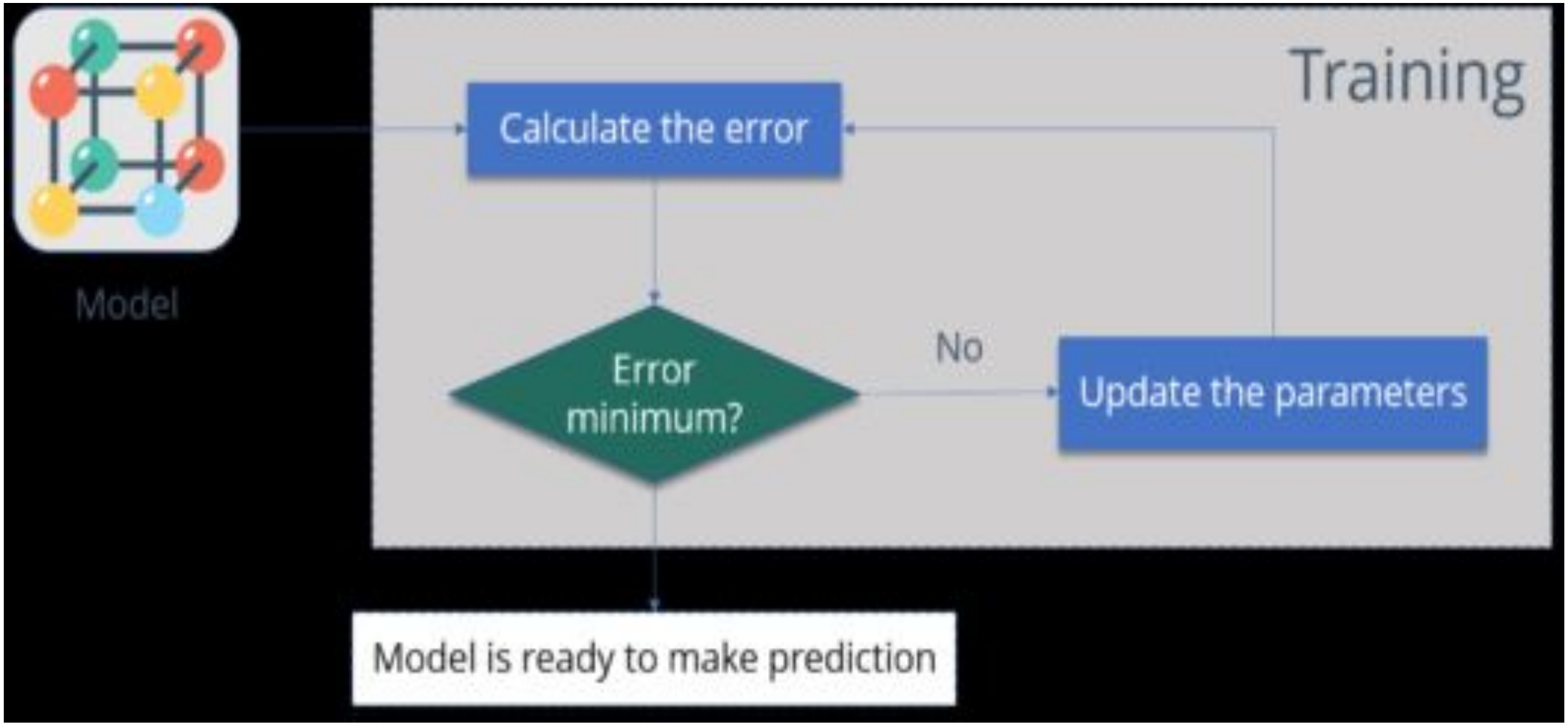


Below are the steps that an artificial neural network follows to **gain maximum accuracy** and **minimize error values**

- **Steps:**
  - **1. Parameter Initialization**
  - **2. Feed Forward Propagation**
  - **3. Back Propagation**



One way to train our model is called as Back propagation. Consider the diagram below:



Let me summarize the steps for you:

Calculate the error – How far is your model output from the actual output.

Minimum Error – Check whether the error is minimized or not.

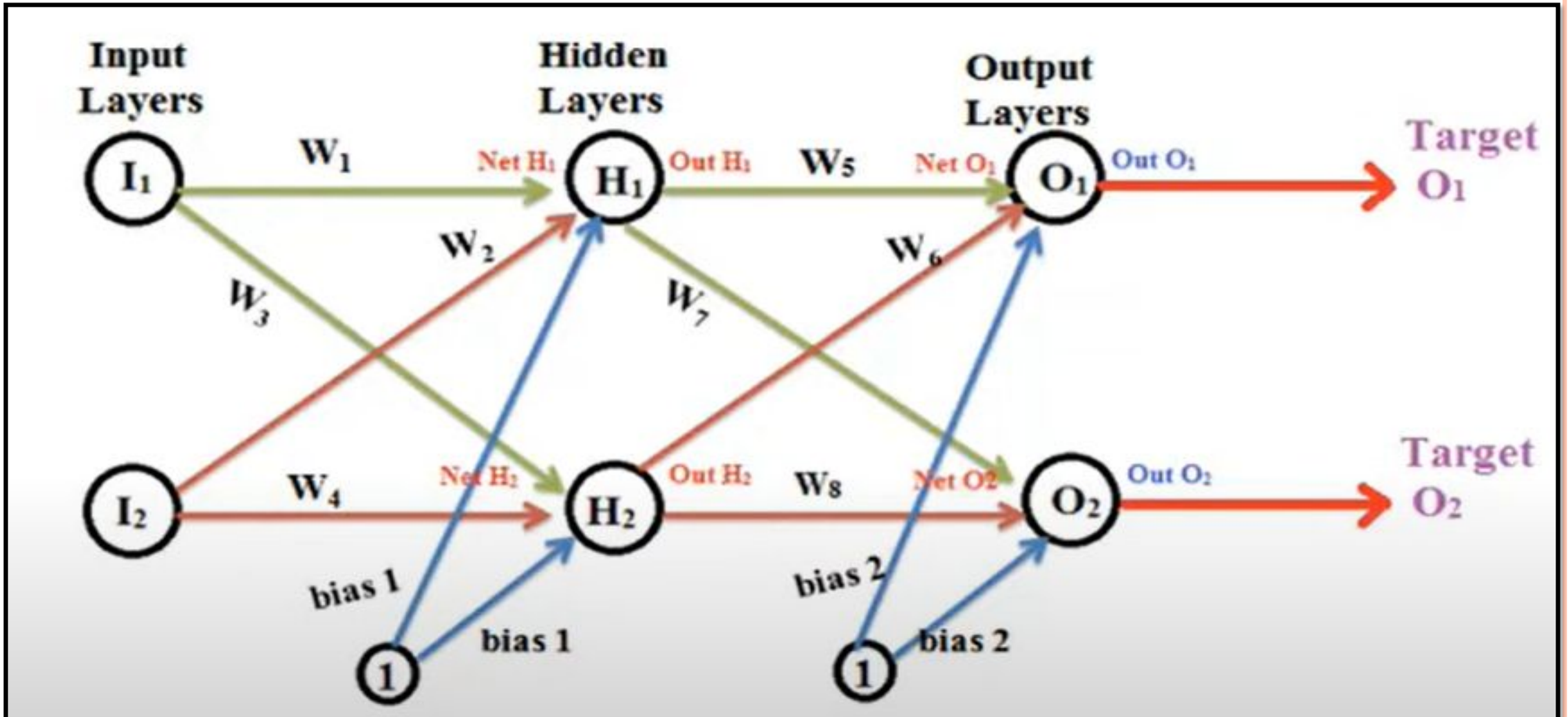
Update the parameters – If the **error is huge** then, **update the parameters (weights and biases).** After that again check the error. Repeat the process until the error becomes minimum.

Model is ready to make a prediction – Once the **error becomes minimum**, you can **feed some inputs to your model and it will produce the output.**



BACK PROPAGATION ALGORIHM

- The below Figure show you the Network: i.e, Feed Forward Network



BACK PROPAGATION ALGORIHM

In the above Figure,

- Input layer with two inputs neurons
  - I1, I2 are the Input Layers
- One hidden layer with two neurons
  - H1, H2 are the Hidden Layers
- Output layer with two neurons
  - O1,O2 are the Output Layers
- Target O1, Target O2 are the Desired Output (or) the Expected Output

- Here I1, I2 are the Input Layers and W1, W2, W3, W4 are the corresponding Weights.
- From the above Network, First we can Calculate the Net H1 and Net H2.

$$net\,H_1 = I_1.W_1 + I_2.W_2 + b_1 \quad .....(1)$$

$$net\,H_2 = I_1.W_3 + I_2.W_4 + b_1 \quad .....(2)$$

BACK PROPAGATION ALGORIHM

- **After Calculating net H1 and net H2. compute Out H1 and Out H2 as follows:**

$$Out\ H_1 = \frac{1}{1+e^{-net H_1}} \quad .....(3)$$

$$Out\ H_2 = \frac{1}{1+e^{-net H_2}} \quad .....(4)$$

- Here we can use the **Activation Function** is the **sigmoid function** ,the values for which it is used in the range, **0 to 1.**
- It is used for models where we have to **predict the probability.** Since the probability of any event lies **between 0 and 1**, the **sigmoid function** is the **right choice.**

BACK PROPAGATION ALGORIHM

- Similarly, we can Calculating net O1 and net O2.

$$net\,O_1 = W_5.out\,H_1 + W_6.out\,H_2 + b_2 \quad .....(5)$$

$$net\,O_2 = W_7.out\,H_1 + W_8.out\,H_2 + b_2 \quad .....(6)$$

- From the above Equations, we can calculate Out O1 and Out O2 as follows:

$$Out\,O_1 = \frac{1}{1+e^{-netO_1}} \quad .....(7)$$

$$Out\,O_2 = \frac{1}{1+e^{-netO2}} \quad .....(8)$$

BACK PROPAGATION ALGORIHM

- Now , Set some Target Values for both Outputs and Calculate Error Outputs based on Target and Calculated Outputs.

$$E_{O_1} = \frac{1}{2}\{Target\ O_1 - Output\ O_1\}^2$$

Here, E_{01} is the Error at Output O1

$$E_{O_2} = \frac{1}{2}\{Target\ O_2 - Output\ O_2\}^2$$

Here, E_{02} is the Error at Output O2

- Next, we can Calculating the Total Error

$$E_T = E_{O_1} + E_{O_2} \quad .....(9)$$

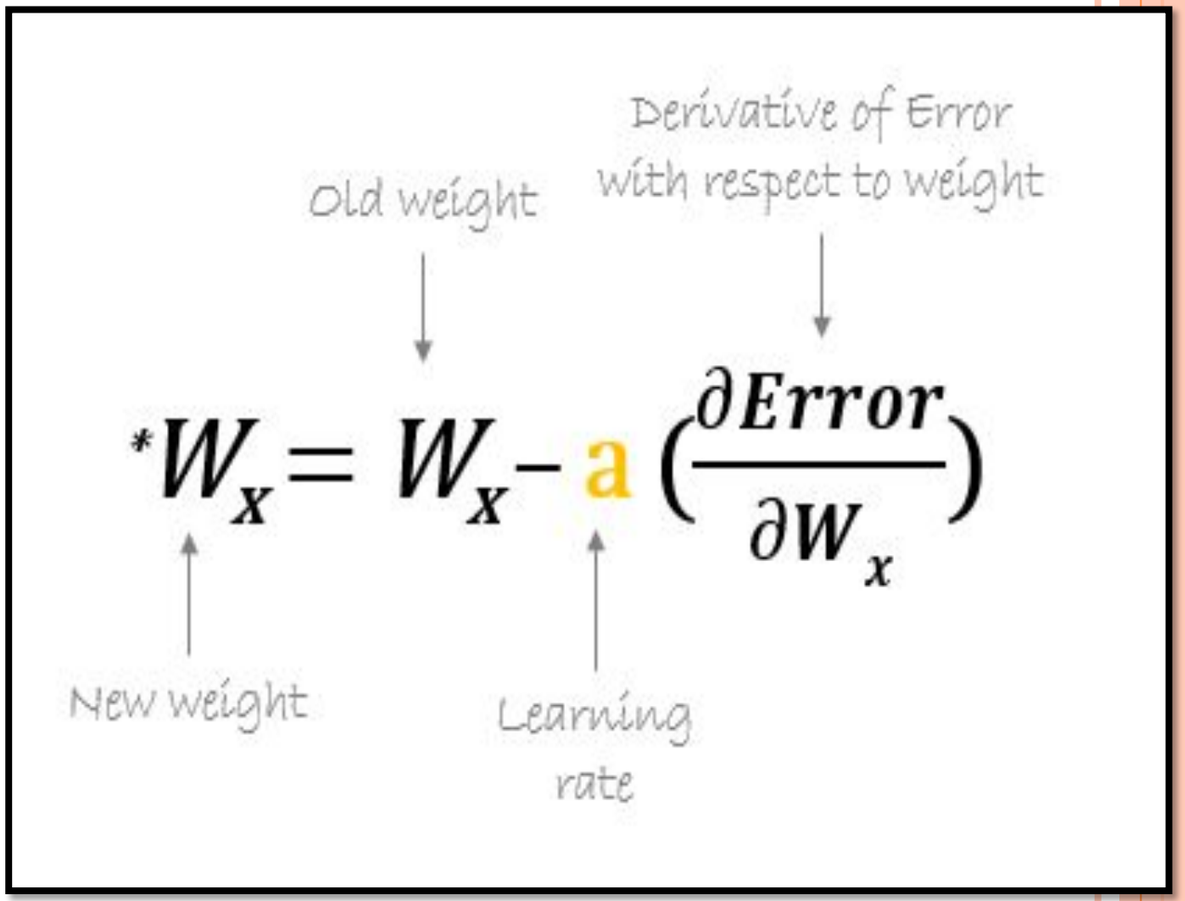
Here, E_T is the Total Error

- Our **main goal** of the **training** is to **reduce the error** or the difference between **prediction and actual output.**
- Suppose there is an **Error occurred** , we will use **" Back Propagation"**

BACK PROPAGATION

- **Back propagation,** short for **"backward propagation of errors",** is a mechanism used to **update the weights** using gradient descent.
- **Gradient descent** is an **iterative optimization algorithm** for **finding the minimum of a function**.
- In our case we want to **minimize the error function**.
- It calculates the **gradient of the error function** with respect to the **neural network's weights**.
- The calculation proceeds **backwards through the network**.

- **The Formula for the Gradient Descent is:**



BACK PROPAGATION

- **2. Back ward Pass:**
- When we **got the Error**. i.e, **Target Output O1 and Actual Output O1.**
- Similarly, **Target Output O2 and Actual Output O2.**
- So that we have to **move Backward** and we have to **adjust the weights** of **W5, W6, W7, W8.**
- Now we have to **Calculate** the **Output Values.**
- If we get the **Expected Output and Actual Outputs are Same, We will stop**.

- Otherwise again **we will back and Update** the **weights** of **W1, W2, W3, W4** also.
- This process continues unless until the **Actual Output and Target Output are the Same.**
- **Update the Weights: Here we can use the Gradient Descent.**
- **i.e, for suppose we have to update the weights : W5**
- **1. For W5:**

$$W_5^+ = W_5 - a\left(\frac{\partial Error}{\partial W_5}\right)$$

BACK PROPAGATION

- Here, **W5** has to be updated w.r.to the **Total Error.**
- So here **first calculate** the

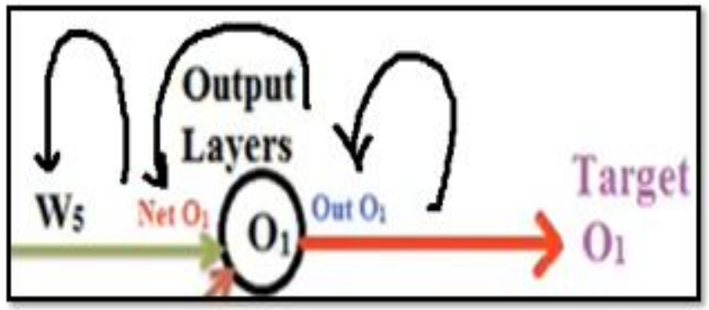
$$\left(\frac{\partial Error}{\partial W_5}\right)$$

- **This can be written as**

$$\frac{\partial E_{Total}}{\partial_{W_5}} = \frac{\partial E_{Total}}{\partial_{OutO_1}} \cdot \frac{\partial_{OutO_1}}{\partial_{netO_1}} \cdot \frac{\partial_{netO_1}}{\partial_{W_5}} \quad \ldots\ldots(10)$$

- So, here will calculate **each and every term separately.**

- Here we can apply the **Chain Rule**.



- **First term,**

$$\frac{\partial E_{Total}}{\partial_{OutO_1}} = \frac{\partial (E_{O_1} + E_{O_2})}{\partial_{OutO_1}} \quad \rightarrow (11)$$

BACK PROPAGATION

- we will written as,

$$\frac{\partial E_{Total}}{\partial_{OutO_1}} = \frac{\frac{1}{2}\{Target\,O_1 - Output\,O_1\}^2 + \frac{1}{2}\{Target\,O_2 - Output\,O_2\}^2}{\partial_{OutO_1}}$$

- **After apply the partial Derivation, we will get :**

$$\frac{\partial E_{Total}}{\partial_{OutO_1}} = \frac{1}{2}.2\{Target\,O_1 - Output\,O_1\}.(-1) + 0$$

- **After Simplifying,**

$$\frac{\partial E_{Total}}{\partial_{OutO_1}} = -\{Target\,O_1 - Output\,O_1\} \ldots\ldots\ldots (12)$$

BACK PROPAGATION

- **Second term,**

$$\frac{\partial (out\ o_1)}{\partial net\ o_1} = \frac{1}{\partial net\ o_1}\left(\frac{1}{1+e^{-net\ o_1}}\right)$$

Assume net $O_1 = x$

$$= \frac{1}{\partial x}\left[\frac{1}{1+e^{-x}}\right]$$

- **Apply the partial Derivation, we will get :**

$$= \frac{1}{\partial x}\left[\frac{1}{1+e^{-x}}\right]$$

$$= \frac{1}{\partial x}\left[(1+e^{-x})^{-1}\right]$$

$$= -1 \cdot (1+e^{-x})^{-1-1} \cdot (-1) \cdot e^{-x}$$

$$= (1+e^{-x})^{-2} \cdot e^{-x} = \frac{e^{-x}}{(1+e^{-x})^{2}}$$

BACK PROPAGATION

- After apply the partial Derivation, we will get :

$$= \frac{1}{(1+e^{-x})} \cdot \frac{1}{(1+e^{-x})} \times e^{-x}$$

$$= \frac{1}{(1+e^{-x})} \cdot \left[1 - \frac{1}{1+e^{-x}}\right]$$

$$= out_{O1} * (1 - out_{O1}) \longrightarrow (13) \qquad \left[\because out_{O1} = \frac{1}{1+e^{-x}}\right]$$

- Third term,

$$\frac{\partial (net_{O1})}{\partial w_5} = \frac{\partial}{\partial w_5}\left[w_5 * out_{H1} + w_6 * out_{H2} + b_2 * 1\right]$$

$$= out_{H1} \longrightarrow (14)$$

- By combining, all these terms

$$\frac{\partial E_{Total}}{\partial w_5} = \frac{\partial E_{Total}}{\partial out_{O1}} * \frac{\partial out_{O1}}{\partial net_{O1}} * \frac{\partial net_{O1}}{\partial w_5}$$

$$= -\left[Target_{O1} - output_{O1}\right] * out_{O1}\left[1 - out_{O1}\right] * out_{H1}$$

BACK PROPAGATION

- After apply the partial Derivation, we will get :

$$= \frac{1}{(1+e^{-x})} \cdot \frac{1}{(1+e^{-x})} \times e^{-x}$$

$$= \frac{1}{(1+e^{-x})} \cdot \left[1 - \frac{1}{1+e^{-x}}\right]$$

$$= out_{O1} * (1 - out_{O1}) \longrightarrow (13) \qquad \left[\because out_{O1} = \frac{1}{1+e^{-x}}\right]$$

- Third term,

$$\frac{\partial (net_{O1})}{\partial w_5} = \frac{\partial}{\partial w_5}\left[w_5 * out_{H1} + w_6 * out_{H2} + b_2 * 1\right]$$

$$= out_{H1} \longrightarrow (14)$$

- By combining, all these terms

$$\frac{\partial E_{Total}}{\partial w_5} = \frac{\partial E_{Total}}{\partial out_{O1}} * \frac{\partial out_{O1}}{\partial net_{O1}} * \frac{\partial net_{O1}}{\partial w_5}$$

$$= -\left[Target_{O1} - output_{O1}\right] * out_{O1}\left[1 - out_{O1}\right] * out_{H1}$$

BACK PROPAGATION

- After Calculating the $\left(\frac{\partial Error}{\partial W_s}\right)$
- Update the Weight , W5 as

$$W_s^+ = W_s - a\left(\frac{\partial Error}{\partial W_s}\right)$$

- Here,
- W5 is the Old Value
- a – is the Learning Rate or Step Rate.

Similarly Update all the Weights W_6 as W_6^+

W_1 as W_1^+

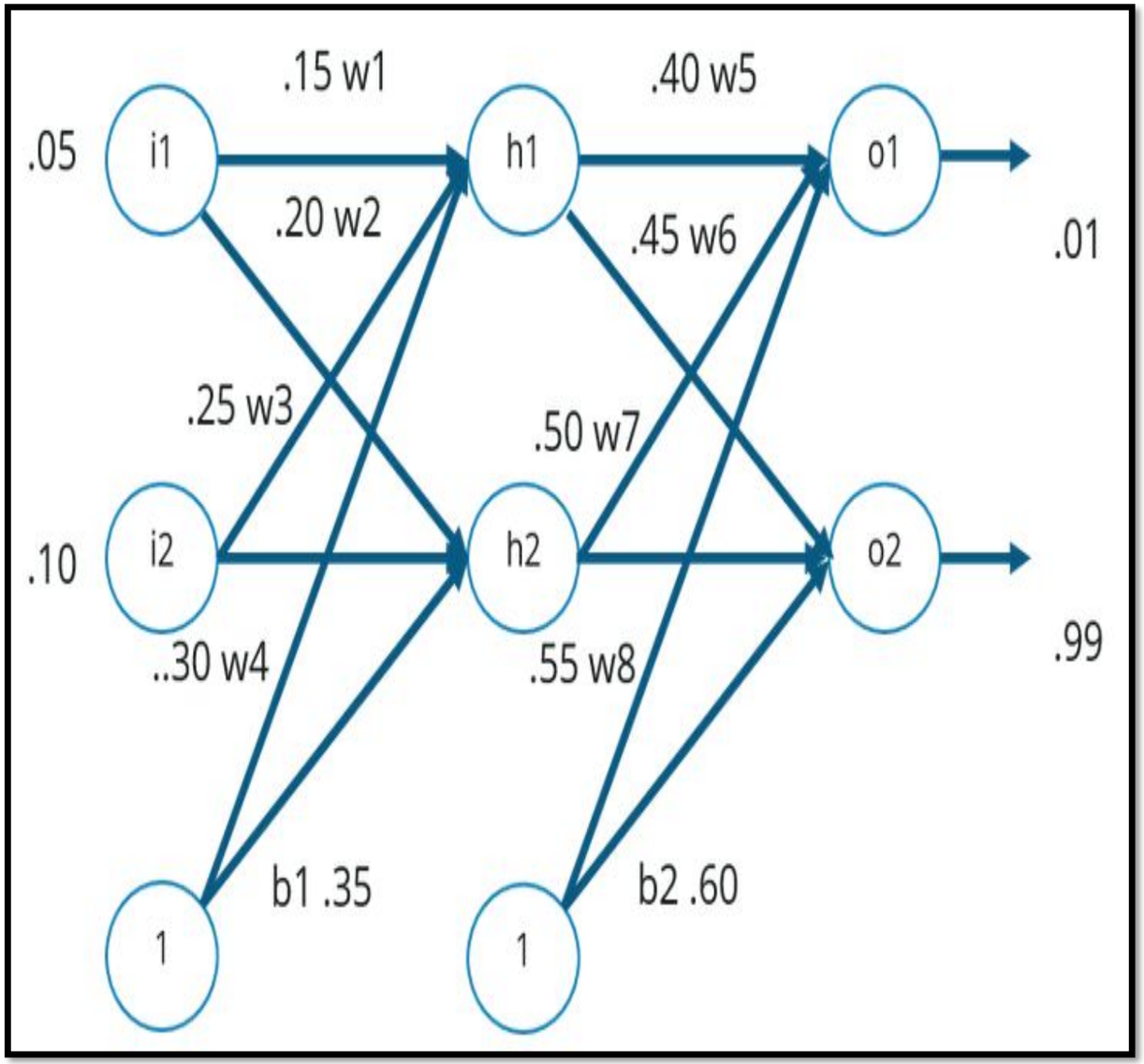
W_2 as W_2^+ etc

Now, with new Updated weights

Calculate new Total Error (E_T) & Repeat the

Procedure until E_T is equal to zero.

Example Problem



The above **network contains the following:**

1. **two inputs : X1 and X2**
2. **two hidden neurons : h1 and h2**
3. **two output neurons : o1 and o2**
4. **two biases : b1 and b2**

Below are the **steps involved in Back propagation:**

Step – 1: Forward Propagation

Step – 2: Backward Propagation

Step – 3: Putting all the values together and calculating the updated weight value

Example Problem

Assume the following values:

$I_1 = 0.05$ $W_1 = 0.15$ $W_5 = 0.4$ $b_1 = 0.35$

$I_2 = 0.10$ $W_2 = 0.2$ $W_6 = 0.45$ $b_2 = 0.6$

$W_3 = 0.25$ $W_7 = 0.5$

$W_4 = 0.3$ $W_8 = 0.55$

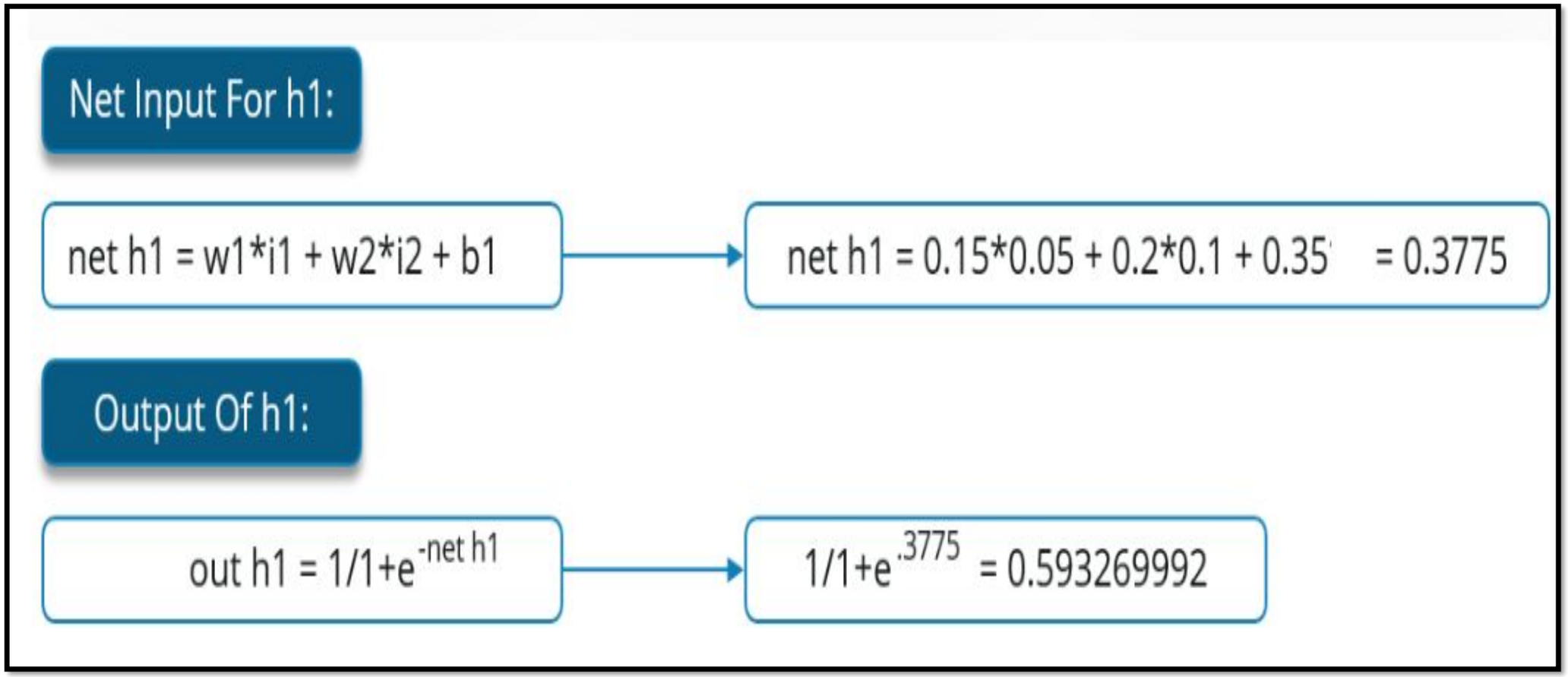
Target $O_1 = 0.01$

Target $O_2 = 0.99$

Example Problem

Step – 1: Forward Propagation :

We will start by **propagating forward**.



Example Problem

Net Input For h2:

$$\text{Net } h2 = i1 * W3 + i2 * W4 + B1$$

$$= 0.05 * 0.25 + 0.10 * 0.30 + 0.35 = 0.3925$$

$$\text{Out } h2 = 1/1+e^{-\text{net } h2}$$

$$= 1/1+e^{-0.3925}$$

$$= 0.59688$$

Output Of h2:

out h2 = 0.596884378

Example Problem

We will **repeat this process** for the **output layer neurons**, using the **output from the hidden layer neurons as inputs**.

Output For o1:

net o1 = w5*out h1 + w6*out h2 + b2 → 0.4*0.593269992 + 0.45*0.596884378 + 0.6 = 1.105905967

Out o1 = $1/1+e^{-\text{net o1}}$ → $1/1+e^{-1.105905967}$ = 0.75136507

Example Problem

Net o2 = w7 * out h1 + w8 * out h2 + b2

= 0.50 * 0.5932 + 0.55 * 0.59688 + 0.60

= 1.22286

Out o2 = $1/1 + e^{-\text{net o2}}$

= $1/ 1+ e^{-1.22286}$

Output For o2:

Out o2 = 0.772928465

Calculating the Total error:

Error For o1:

$E_{o1} = \Sigma 1/2(target - output)^2$ → $\frac{1}{2}(0.01 - 0.75136507)^2 = 0.274811083$

Error For o2:

E o2 = 0.023560026

Total Error:

$E_{total} = E_{o1} + E_{o2}$ → 0.274811083 + 0.023560026 = 0.298371109

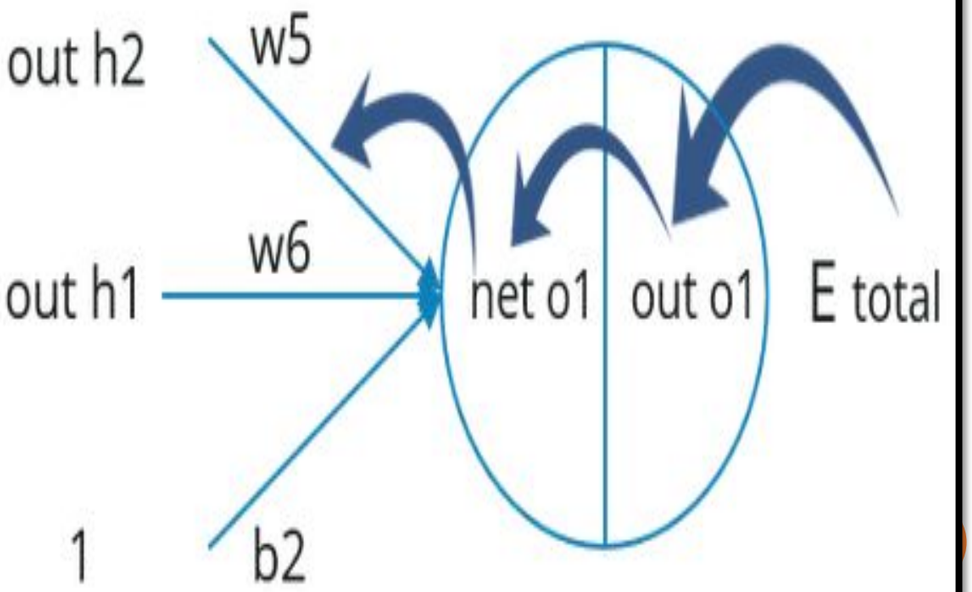
Example Problem

Step – 2: Backward Propagation

Now, we will **propagate backwards**. This way we will try to **reduce the error** by **changing the values of weights and biases.**

Consider **W5,** we will calculate the rate of change of error w.r.t **change in weight W5.**

$$\frac{\delta Etotal}{\delta w5} = \frac{\delta Etotal}{\delta out\ o1} * \frac{\delta out\ o1}{\delta net\ o1} * \frac{\delta net\ o1}{\delta w5}$$



Example Problem

Calculate Each and Every Term:

First Term:

$$E_{total} = 1/2(\text{target o1} - \text{out o1})^2 + 1/2(\text{target o2} - \text{out o2})^2$$

$$\frac{\delta Etotal}{\delta out\ o1} = -(\text{target o1} - \text{out o1}) = -(0.01 - 0.75136507) = 0.74136507$$

Calculate Second Term:

$$\text{out o1} = 1/1+e^{-neto1}$$

$$\frac{\delta out\ o1}{\delta net\ o1} = \text{out o1 (1 - out o1)} = 0.75136507\ (1 - 0.75136507) = 0.186815602$$

Calculate Third Term:

$$\frac{\delta net\ o1}{\delta w5} = \text{out h1} = 0.593269992$$

Example Problem

Step – 3: Putting all the values together and **calculating the updated weight value**

$$\frac{\delta Etotal}{\delta w5} = \frac{\delta Etotal}{\delta out\ o1} * \frac{\delta out\ o1}{\delta net\ o1} * \frac{\delta net\ o1}{\delta w5}$$

→ 0.082167041

$$= -[0.01 - 0.7513] * 0.7513[1-0.7513] * 0.5932$$

$$= 0.082$$

Example Problem

Let's calculate the updated value of W5:

$$w5^{+} = w5 - n\frac{\delta Etotal}{\delta w5}$$

$$w5^{+} = 0.4 - 0.5 * 0.082167041$$

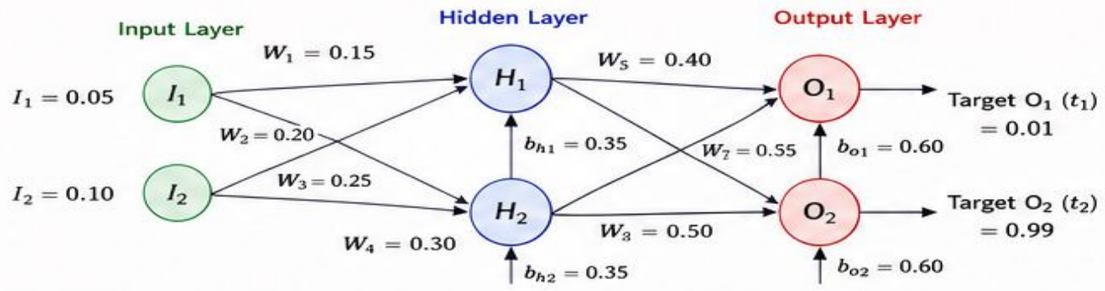
Updated w5 → 0.35891648

- Similarly, we can **calculate the other weight values as well.**
- After that we will **again propagate forward** and **calculate the output.** Again, **we will calculate the error.**
- If the **error is minimum** we **will stop right there**, else we will **again propagate backwards** and **update the weight values.**
- This **process will keep on repeating until error becomes minimum.**

BACK PROPAGATION ALGORITHM – EXAMPLE

Network: 2-2-2 Neural Network (2 Inputs, 2 Hidden Neurons, 2 Outputs)

1. NETWORK ARCHITECTURE & INITIAL VALUES



Learning rate (η) = 0.5 Activation Function : Sigmoid $f(x) = \frac{1}{1+e^{-x}}$

2. FORWARD PROPAGATION

Step 1 : Hidden layer net inputs

$$net_{h1} = I_1 W_1 + I_2 W_3 + b_{h1}$$
$$= (0.05)(0.15) + (0.10)(0.25) + 0.35$$
$$= 0.0075 + 0.0250 + 0.35 = 0.3825$$

$$net_{h2} = I_1 W_2 + I_2 W_4 + b_{h2}$$
$$= (0.05)(0.20) + (0.10)(0.30) + 0.35$$
$$= 0.0100 + 0.0300 + 0.35 = 0.3900$$

Step 2 : Hidden layer outputs

$$out_{h1} = f(net_{h1}) = \frac{1}{1+e^{-0.3825}} = 0.5943$$
$$out_{h2} = f(net_{h2}) = \frac{1}{1+e^{-0.3900}} = 0.5963$$

Step 3 : Output layer net inputs

$$net_{o1} = out_{h1} W_5 + out_{h2} W_7 + b_{o1}$$
$$= (0.5943)(0.40) + (0.5963)(0.50) + 0.60$$
$$= 0.23772 + 0.29815 + 0.60 = 1.13587$$

$$net_{o2} = out_{h1} W_6 + out_{h2} W_8 + b_{o2}$$
$$= (0.5943)(0.45) + (0.5963)(0.55) + 0.60$$
$$= 0.26744 + 0.32797 + 0.60 = 1.19541$$

Step 4 : Output layer outputs

$$out_{o1} = f(net_{o1}) = \frac{1}{1+e^{-1.13587}} = 0.7579$$
$$out_{o2} = f(net_{o2}) = \frac{1}{1+e^{-1.19541}} = 0.7671$$

3. LOSS (ERROR) CALCULATION

Error at O_1 :
$$E_{o1} = \frac{1}{2}(t_1 - out_{o1})^2 = \frac{1}{2}(0.01 - 0.7579)^2 = 0.2793$$

Error at O_2 :
$$E_{o2} = \frac{1}{2}(t_2 - out_{o2})^2 = \frac{1}{2}(0.99 - 0.7671)^2 = 0.0248$$

Total Error :
$$E_T = E_{o1} + E_{o2} = 0.2793 + 0.0248 = 0.3041$$

4. BACKWARD PROPAGATION – OUTPUT LAYER

Step 1 : Error terms (delta)

$$\delta_{o1} = (t_1 - out_{o1}) \times out_{o1}(1 - out_{o1})$$
$$= (0.01 - 0.7579)(0.7579)(1 - 0.7579)$$
$$= (-0.7479)(0.7579)(0.2421)$$
$$= -0.1369$$

$$\delta_{o2} = (t_2 - out_{o2}) \times out_{o2}(1 - out_{o2})$$
$$= (0.99 - 0.7671)(0.7671)(1 - 0.7671)$$
$$= (0.2229)(0.7671)(0.2329)$$
$$= 0.0399$$

Step 2 : Gradients (Hidden → Output Weights)

$$\frac{\partial E}{\partial W_5} = \delta_{o1} \times out_{h1} = (-0.1369)(0.5943) = -0.0813$$
$$\frac{\partial E}{\partial W_7} = \delta_{o1} \times out_{h2} = (-0.1369)(0.5963) = -0.0817$$
$$\frac{\partial E}{\partial W_6} = \delta_{o2} \times out_{h1} = (0.0399)(0.5943) = 0.0237$$
$$\frac{\partial E}{\partial W_8} = \delta_{o2} \times out_{h2} = (0.0399)(0.5963) = 0.0238$$

Step 3 : Update Weights (New = Old + η × gradient)

$$W_5^{new} = 0.40 + 0.5(-0.0813) = 0.3594$$
$$W_7^{new} = 0.50 + 0.5(-0.0817) = 0.4592$$
$$W_2^{new} = 0.45 + 0.5(0.0237) = 0.4618$$
$$W_8^{new} = 0.55 + 0.5(0.0238) = 0.5619$$

5. BACKWARD PROPAGATION – HIDDEN LAYER

Step 1 : Error terms (delta) for hidden neurons

$$\delta_{h1} = out_{h1}(1 - out_{h1})(W_5\delta_{o1} + W_6\delta_{o2})$$
$$= (0.5943)(1 - 0.5943)[(0.40)(-0.1369) + (0.45)(0.0399)]$$
$$= (0.5943)(0.4057)(-0.05476 + 0.01796)$$
$$= (0.5943)(0.4057)(-0.03680) = -0.00896$$

$$\delta_{h2} = out_{h2}(1 - out_{h2})(W_7\delta_{o1} + W_8\delta_{o2})$$
$$= (0.5963)(1 - 0.5963)[(0.50)(-0.1369) + (0.55)(0.0399)]$$
$$= (0.5963)(0.4037)(-0.06845 + 0.02195)$$
$$= (0.5963)(0.4037)(-0.04650) = -0.01113$$

Step 2 : Gradients (Input → Hidden Weights)

$$\frac{\partial E}{\partial W_1} = \delta_{h1} \times I_1 = (-0.00896)(0.05) = -0.000448$$
$$\frac{\partial E}{\partial W_3} = \delta_{h1} \times I_2 = (-0.00896)(0.10) = -0.000896$$
$$\frac{\partial E}{\partial W_2} = \delta_{h2} \times I_1 = (-0.01113)(0.05) = -0.000557$$
$$\frac{\partial E}{\partial W_4} = \delta_{h2} \times I_2 = (-0.01113)(0.10) = -0.001113$$

Step 3 : Update Weights (New = Old + η × gradient)

$$W_1^{new} = 0.15 + 0.5(-0.000448) = 0.14978$$
$$W_3^{new} = 0.25 + 0.5(-0.000896) = 0.24955$$
$$W_2^{new} = 0.20 + 0.5(-0.000557) = 0.19972$$
$$W_4^{new} = 0.30 + 0.5(-0.001113) = 0.29944$$

Bias Update for Hidden Layer

$$b_{h1}^{new} = 0.35 + 0.5(-0.00896) = 0.34552$$
$$b_{h2}^{new} = 0.35 + 0.5(-0.01113) = 0.34444$$

Bias Update for Output Layer

$$b_{o1}^{new} = 0.60 + 0.5(-0.1369) = 0.53155$$
$$b_{o2}^{new} = 0.60 + 0.5(0.0399) = 0.61997$$

6. SUMMARY AFTER FIRST ITERATION

Parameter	Old Value	New Value
W_1	0.15	0.14978
W_2	0.20	0.19972
W_3	0.25	0.24955
W_4	0.30	0.29944
W_5	0.40	0.35940
W_6	0.45	0.46180
W_7	0.50	0.45920
W_8	0.55	0.56190
b_{h1}	0.35	0.34552
b_{h2}	0.35	0.34444
b_{o1}	0.60	0.53155
b_{o2}	0.60	0.61997

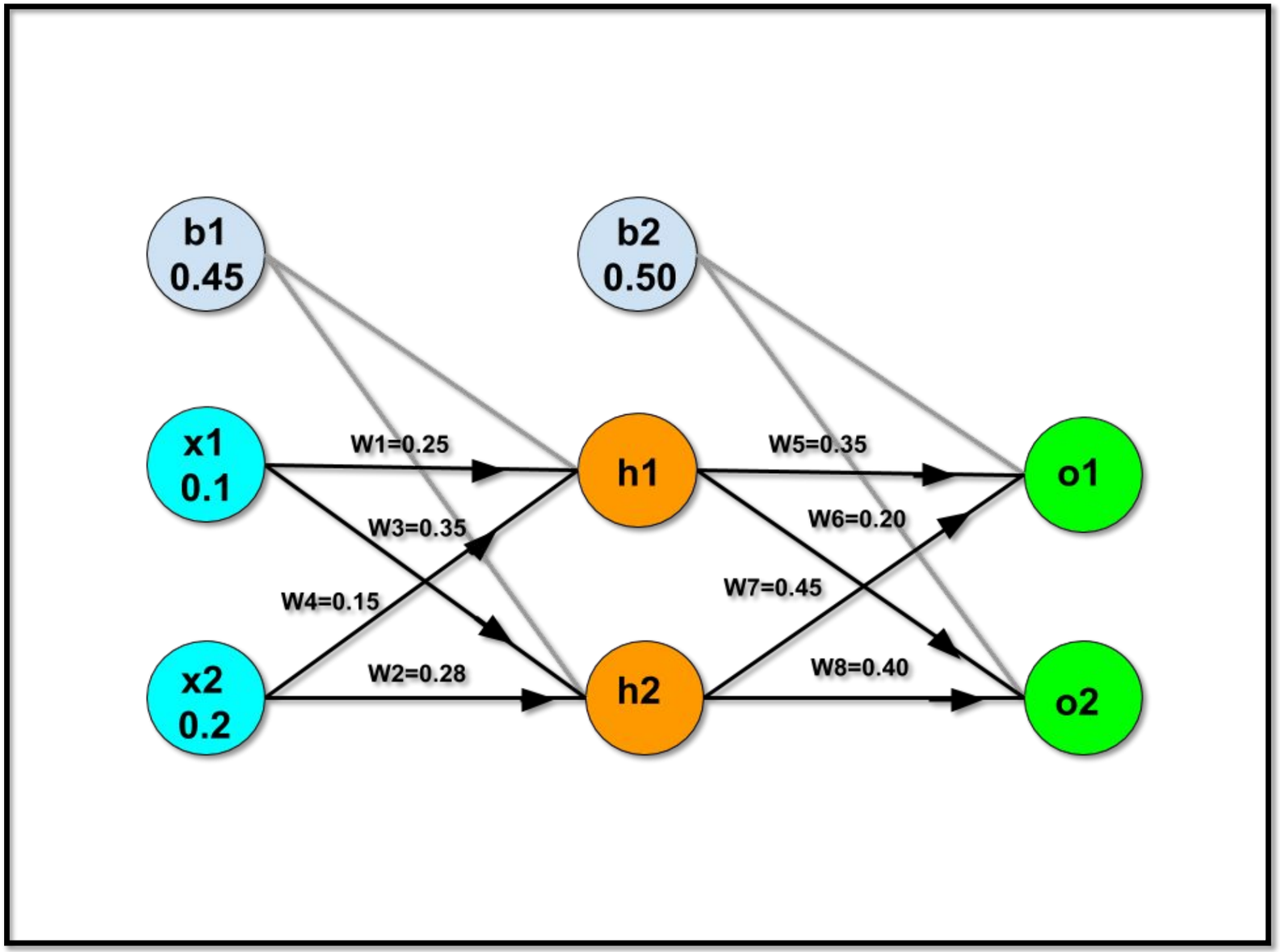
7. REPEAT

Use the updated weights and biases. Repeat steps 2 to 6 for all training samples for multiple epochs until the total error (E_T) becomes minimum or convergence is achieved.

KEY POINTS

- Forward pass computes outputs.
- Loss is calculated using target values.
- Backward pass computes error terms (delta).
- Gradients are calculated.
- Weights and biases are updated.
- Process is repeated until error is minimum.

EXAMPLE-2



The correct output from output node *o1* and *o2* be *y1* and *y2* respectively. Let's assume the value of *y1 = 0.05* and the value of *y2 = 0.95* which are the correct outputs labeled for the given inputs.

Advantages

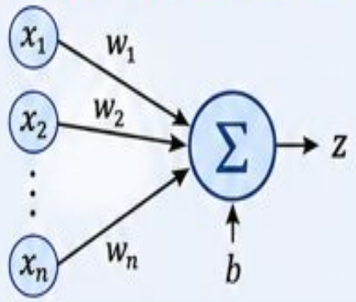
- Efficient training of deep neural networks.
- Reduces prediction error.
- Supports multiple hidden layers.
- Works with various activation functions.
- Enables learning of complex features.
- Forms the foundation of modern deep learning models.

Applications

- Image Classification
- Object Detection
- Face Recognition
- Speech Recognition
- Natural Language Processing (NLP)
- Machine Translation
- Medical Image Analysis
- Fraud Detection
- Autonomous Driving
- Recommendation Systems

IMPORTANT MATHEMATICAL EQUATIONS

1. WEIGHTED SUM

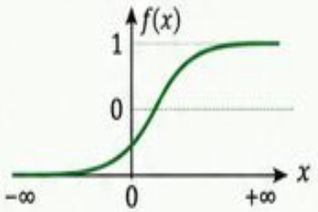


$$Z = WX + b$$

- Z = Weighted sum (net input)
- W = Weight matrix / vector
- X = Input vector
- b = Bias vector

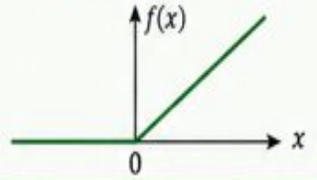
2. ACTIVATION FUNCTION

- **SIGMOID**



$$f(x) = \frac{1}{1 + e^{-x}}$$

- **ReLU**



$$f(x) = \max(0, x)$$

3. MEAN SQUARED ERROR



$$L = \frac{1}{2}(y - \hat{y})^2$$

- y = Actual output
- $\hat{y}$ = Predicted output

4. GRADIENT ∇

$$\frac{\partial L}{\partial W}$$

Gradient of loss (L) with respect to weights (W)

5. WEIGHT UPDATE

$$W = W - \eta \frac{\partial L}{\partial W}$$

- η = Learning rate
- $\frac{\partial L}{\partial W}$ = Gradient of loss with respect to W

6. BIAS UPDATE

$$b = b - \eta \frac{\partial L}{\partial b}$$

- η = Learning rate
- $\frac{\partial L}{\partial b}$ = Gradient of loss with respect to b

Where:

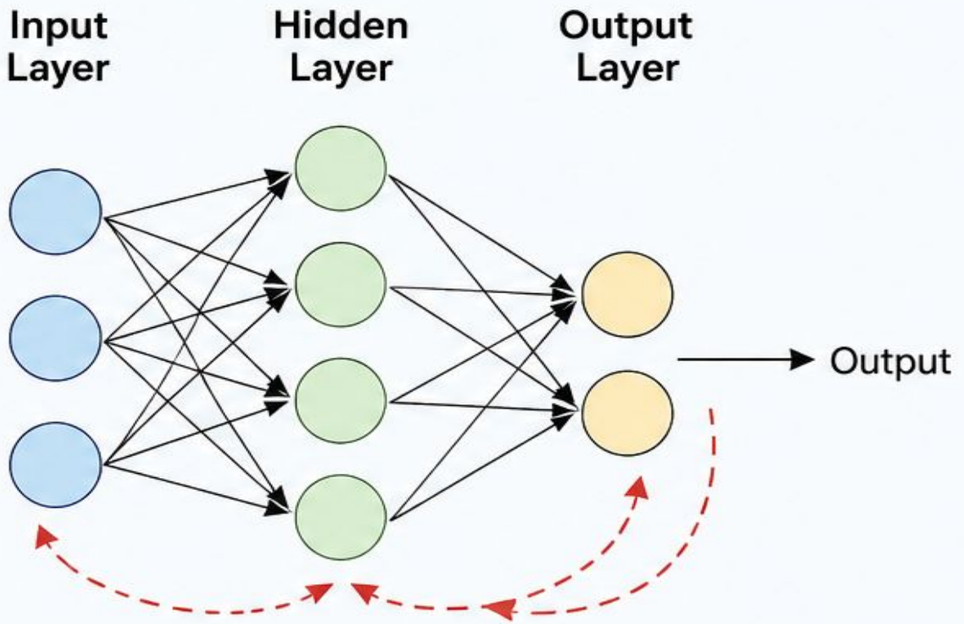
- W = Weights
- b = Bias
- X = Input
- Z = Net input (weighted sum)
- $f(x)$ = Activation function
- L = Loss
- η = Learning rate

BACK PROPAGATION

THANK YOU!

Thank you for your attention.
I hope this session on **Back Propagation** was insightful and inspiring.

Learning today,
Building intelligent solutions for tomorrow.



Back Propagation

Error is propagated backward and weights are updated to minimize loss.